

ACC: Compiling Agent Trajectories for Long-Context Training

Qisheng Su^{1,2}, Zhen Fang¹, Shiting Huang¹, Yu Zeng¹, Yiming Zhao¹,
Kou Shi¹, Ziao Zhang¹, Lin Chen¹, Zehui Chen¹, Lijun Wu³, Feng Zhao^{1*}

¹MoE Key Lab of BIPC, University of Science and Technology of China

²Shanghai Innovation Institute

³Shanghai AI Laboratory

nicksu@mail.ustc.edu.cn fzhao956@ustc.edu.cn

 **Dataset:** <https://huggingface.co/datasets/groundhogLLM/ACC-dataset>

 **Checkpoint:** <https://huggingface.co/groundhogLLM/ACC-Qwen3-30B-A3B>

Abstract

Recent development of agents has renewed demand for long-context reasoning capacity of LLMs. However, training LLMs for this capacity requires costly long-document curation or heuristic context synthesis. We observe that agents produce massive trajectories when solving problems, invoking tools and receiving environment observations across many turns. The evidence needed to answer the original question is thus scattered throughout these turns, requiring integration of distant context segments. Nevertheless, standard agent SFT masks tool responses and only trains turn-level tool selection, creating a supervision blind spot where these scattered signals go unused. We propose Agent Context Compilation (ACC), which converts trajectories from search, software engineering, and database querying agents into long-context QA pairs that combine the original question with tool responses and environment observations gathered across multiple turns, training the model to answer directly without tool use. This makes the dependencies between the question and the evidence explicit, enabling direct supervision of long-context reasoning over distant segments without additional annotation. ACC is a simple but effective approach that can be combined with any existing long-context extension or training method, providing scalable supervised fine-tuning data. We validate ACC on long-range dependency modeling tasks through MRCR and GraphWalks, challenging benchmarks requiring cross-turn coreference resolution and graph traversal over extended contexts. Training Qwen3-30B-A3B with ACC achieves 68.3 on MRCR (+18.1) and 77.5 on GraphWalks (+7.6), results comparable to Qwen3-235B-A22B, while preserving general capabilities on GPQA, MMLU-Pro, AIME, and IFEval. Further mechanism analysis reveals that the ACC-trained model exhibits task-adaptive attention restructuring and expert specialization. Dataset and checkpoints are released publicly.

1 Introduction

Recently, the rise of agents has brought fresh attention to long-context reasoning for LLMs[15, 2, 6, 16], since agents work through many turns of tool calls and models need to handle increasingly long inputs. However, conventional training of LLMs for this capacity relies on costly long-document curation or heuristic context synthesis. Curating annotated long documents requires precise evidence labeling and intensive quality filtering. Heuristic synthesis gathers contexts without the complex

*Corresponding Author

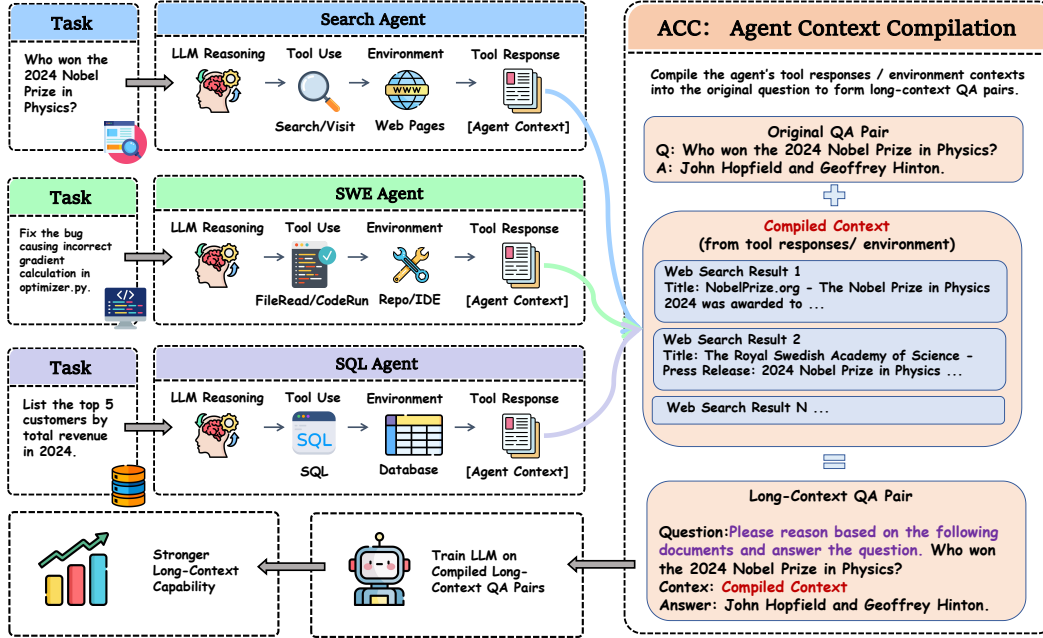


Figure 1: **Overview of ACC.** Multi-turn agent trajectories (Search, SWE, SQL) are compiled into long-context QA pairs by assembling tool responses and environment contexts.

dependencies that actual problem solving creates. These limitations severely restrict scalable training for long-span reasoning and motivate the exploration of alternative supervision sources.

Agents produce massive multi-turn trajectories when solving problems, invoking tools and receiving tool responses across many turns. The evidence needed to answer the original question is scattered throughout these turns, requiring integration of distant context segments. Although these trajectories can be directly used for supervised fine-tuning, standard practice masks out tool responses and only supervises turn-level tool selection. This creates a supervision blind spot that leaves scattered evidence signals unused and severely limits the development of long-context capabilities.

To address this, we propose **Agent Context Compilation (ACC)**, which converts agent trajectories into long-context training data without additional human annotation. By assembling the original question with tool responses and environment observations gathered across multiple turns into one context, ACC makes the dependencies between the question and scattered evidence explicit, enabling direct supervision of long-context reasoning without additional annotation. **ACC is a simple but effective approach that can be combined with existing long-context extension or training method, providing scalable supervised fine-tuning data.** Figure 1 illustrates the ACC pipeline.

We apply ACC to three representative agent classes including search agents that retrieve web pages to answer complex questions, SWE agents that inspect source files to resolve issues, and SQL agents that query relational tables for structured analytics. In each case, we compile **answer-verified trajectories** into long-context training pairs, taking the answer directly from the final output without additional human annotation.

We validate ACC on long-range dependency modeling tasks through MRCR and GraphWalks[14], two particularly challenging benchmarks requiring cross-turn coreference resolution and graph traversal over extended contexts. Training Qwen3-30B-A3B with ACC achieves 68.3 on MRCR (+18.1) and 77.5 on GraphWalks (+7.6), results comparable to Qwen3-235B-A22B, while preserving general capabilities on GPQA, MMLU-Pro, AIME, and IFEval. Mechanism analysis further reveals that the ACC-trained model exhibits task-adaptive attention restructuring and expert specialization, reflecting flexible adaptation to distinct long-range reasoning demands.

Contributions. Our main contributions are summarized as follows. (1) We propose Agent Context Compilation (ACC), a method that converts multi-turn agent trajectories into long-context training QAs. (2) We show that the ACC-trained Qwen3-30B-A3B achieves results comparable to Qwen3-235B-A22B on long-range dependency modeling benchmarks including MRCR and GraphWalks,

while preserving general capabilities. (3) Through mechanism analysis, we observe task-adaptive attention restructuring and expert specialization emerging after ACC training, suggesting that the acquired long-range capacity manifests as flexible, task-specific patterns.

2 Related Work

2.1 Long-Context Capacity Evaluation

Evaluating long-context capabilities has evolved significantly. Early benchmarks such as NIAH [9] tested surface-level retrieval by embedding specific facts within distractor text. RULER [7] extended this with variable tracking, aggregation, and multi-hop reasoning tasks. LongBench [3] introduced diverse real-world tasks including QA, summarization, and code understanding. However, performance on these benchmarks has largely saturated, as they mainly test localized retrieval or single-turn reasoning within long contexts. Classic benchmarks such as Musique [22] and NarrativeQA [10] further targeted multi-hop reasoning and long-document narrative understanding. More recently, OpenAI released MRCR (Multi-Round Coreference Resolution) and GraphWalks [14] as direct tests of long-range dependency modeling. By requiring cross-turn coreference resolution and graph traversal over extended contexts, they are substantially harder than prior single-turn or retrieval tasks, and have become standard benchmarks for mainstream large model releases.

2.2 Long-Context Extension and Training

Recent efforts to improve long-context capabilities generally fall into four categories. First, pre-training methods modify position embeddings or attention mechanisms. MrRoPe [21] applies RoPE interpolation and NTK-aware frequency scaling to broaden the context window. ROPE++ [12] reuses the discarded imaginary component of RoPE’s complex form to build parallel attention heads for improved length extrapolation. Native Sparse Attention [28] and Mamba-3 [11] reduce complexity through sparse and linear attention. Second, some works focus on constructing high-quality long documents for pre-training data. Longwanjuan [13] filters texts by coherence, cohesion, and complexity. LiteLong [8] leverages book taxonomies and multi-agent debate for corpora retrieval and concatenation. Quest [19] predicts possible questions and clusters core keywords to stitch short documents. These methods synthesize long texts rather than post-training QA pairs. Third, post-training recipes combine synthetic data with RL. longRLVR [5] generates QA pairs with precise evidence block annotations from long texts. LongPO [4] extracts key short chunks to build short-long preference pairs and applies short-to-long KL constraints in DPO. LoongRL [23] proposes KeyChain to insert irrelevant documents for hard long-context synthesis and stabilizes GRPO with rule rewards and no entropy term. Fourth, employ agent frameworks at inference time to manage long-context memory. QwenLong-L1.5 [18] cleans multi-source documents, builds knowledge graphs, and applies AEPO for dynamic entropy control. MemAgent [27] mixes irrelevant HotpotQA documents and uses Multi-Conv DAPO to decompose long questions into multi independent conversations with memory updates. Our work differs by using agent trajectories as a direct data source for long-context reasoning training, rather than modifying architectures, synthesizing pre-training documents, or relying on complex post-training RL pipelines.

3 Method

3.1 The Supervision Blind Spot of Agent SFT

Standard agent SFT masks all tool responses (observations) and only supervises turn-level reasoning and actions. The model therefore never learns to integrate evidence scattered across multiple turns.

An agent trajectory consists of $k - 1$ interaction turns followed by a final answer turn

$$\tau = (q, (r_1, a_1, o_1), \dots, (r_{k-1}, a_{k-1}, o_{k-1}), (r_k, y)),$$

where r_t is reasoning, a_t is action, o_t is tool response (observation), and (r_k, y) is the final reasoning-answer pair. The history up to turn t is $\mathcal{H}_{<t} = (r_1, a_1, o_1, \dots, r_{t-1}, a_{t-1}, o_{t-1})^2$. Tool responses are masked from the loss and only model-generated tokens are supervised.

²We present interleaved reasoning traces for clarity. Non-interleaved variants do not affect our conclusions.

Formally, the standard objective is

$$\mathcal{L}_{\text{agent}} = - \sum_{t=1}^k \sum_{j \in \mathcal{I}_t} \log P(\text{token}_j \mid \mathcal{H}_{<t}, \text{token}_{<j}), \quad (1)$$

where $\mathcal{I}_t = r_t \cup a_t$ for $t < k$ and $\mathcal{I}_k = r_k \cup y$.

Grouping Eq. (1) by turn reveals its structure

$$\mathcal{L}_{\text{agent}} = \underbrace{\sum_{t=1}^{k-1} \left(- \sum_{j \in r_t \cup a_t} \log P(\text{token}_j \mid \mathcal{H}_{<t}, \text{token}_{<j}) \right)}_{\text{local next-tool selection}} + \underbrace{\left(- \sum_{j \in r_k \cup y} \log P(\text{token}_j \mid \mathcal{H}_{<k}, \text{token}_{<j}) \right)}_{\text{final answer prediction}}. \quad (2)$$

The first $k - 1$ terms supervise only local reasoning and tool selection at each turn. Consider a token in tool response o_t at turn $t < k$. Excluded from the loss, it receives gradients only indirectly through subsequent unmasked tokens. The dominant signal flows along a **short path** to the next action a_{t+1} , where o_t lies in the immediate context. Any gradient relevant to the final answer y must back-propagate through a **long chain** of intermediate turns to reach o_t , and is heavily weakened. Consequently, these intermediate turns act as a **supervision filter**, so o_t is updated primarily to support local action prediction, ignoring answer-relevant features unless they also serve local needs. This is the supervision blind spot of agent SFT.

3.2 Agent Context Compilation

ACC solves this problem by gathering all evidence into one long context C and training the model to write a reasoning trace r and final answer y directly from the question q and context C . The new training objective is

$$\mathcal{L}_{\text{ACC}} = - \sum_{j \in r \cup y} \log P(\text{token}_j \mid q, C, \text{token}_{<j}). \quad (3)$$

Unlike Eq. (1), this objective contains no intermediate action terms, so the final answer supervision reaches every evidence token directly without being filtered through turn-level tool selection. The model therefore learns to integrate scattered evidence into a global answer instead of merely optimizing local next-tool selection.

Given a set of **answer-verified trajectories** $\mathcal{T} = \{\tau_1, \dots, \tau_N\}$, ACC converts each trajectory into a training example

$$\tau_i \mapsto (x_i, y_i, r_i),$$

producing a dataset $\mathcal{D} = \{(x_i, y_i, r_i)\}_{i=1}^M$. Here $x_i = (q_i, C_i)$ combines the original query with the compiled context, y_i is the final answer from the trajectory, and r_i is its reasoning trace.

3.3 Context Construction

For each trajectory we extract structured evidence pieces $\text{Evi}(\tau) = [e_1, \dots, e_m]$ such that the aggregated context alone suffices to answer q without tool use. For **search trajectories** we extract the full text of visited pages and include unvisited candidate results as distractors. For **SWE trajectories** we extract files involved in the correct patch and include additional context files inspected during debugging as distractors. For **SQL trajectories** we extract the complete contents of all tables queried during the trajectory.

To increase task difficulty, we apply a random permutation π over $\{1, \dots, m\}$ and concatenate the pieces into a compiled context

$$C_i = \text{Concat}(e_{\pi(1)}, e_{\pi(2)}, \dots, e_{\pi(m)}), \quad \text{with } |C_i| \leq B, \quad (4)$$

where B is the token budget. Because evidence pieces are self-contained, shuffling forces the model to locate relevant information via semantic association rather than sequential position.

Answer-verified trajectories contain correct answers but lack explicit reasoning traces. We employ **DeepSeek-V3.2-Thinking** to generate candidate rationales and retain only those that lead to the

correct answer y_i . For Search and SQL, these traces are obtained via direct rollout, with pass rates near 100% and 50%, respectively. For SWE, direct rollout achieves only near 10% accuracy, which is too low to scale. We therefore synthesize answer-conditioned reasoning traces from the compiled context and verified patch instead, with all outputs retained (100% yield, see Appendix F). The final training example is the triple (x_i, y_i, r_i) , where $x_i = (q_i, C_i)$ and r_i is the retained reasoning trace.

Search Agent Trajectory Compilation Example

Original Question-Answer Pair:
Question: Which track, credited as a cover of the French new-wave group, appears as a European-bonus track on a French symphonic black-metal band’s early-2000s album, is featured as a roughly five-minute live performance on that group’s mid-1990s live album recorded for radio, and is listed as the third song on Disc One of the group’s late-1990s live collection?
Groundtruth: Les Tzars

Original Agentic Trajectory (abridged):

```

Turn 1 (Search): "French symphonic black metal Indochine
                cover European bonus track"
Result: [Doc A] Anorexia Nervosa - Redemption Process...
        8. Les Tzars (Indochine Cover)... limited press...
        [Doc B] Indochine is a French new wave band...
        [Doc C] Eating disorders treatment and recovery...
-> (Visit) Doc A

Turn 2 (Search): "Indochine live album 1994 radio Les Tzars"
Result: [Doc D] Radio Indochine is the second live album...
        released in 1994... Les tzars - 5:12...
        [Doc E] Indochine au Zenith 1986 live album...
        [Doc F] Neuropsychological assessment of ED...
-> (Visit) Doc D

Turn 3 (Search): "Indochine Indo Live 1997 Disc One track"
Result: [Doc G] INDO LIVE is the third live album...
        released in 1997... Disc One... Les tzars...
        [Doc H] Indochine 13 album 2017...
        [Doc I] Hiroshima mon amour 1959 film...
-> (Visit) Doc G

Turn 4 (Answer): Les Tzars ✓

```

ACC-Compiled Question-Answer Pair (abridged):

Question: Please reason based on the following documents and answer the question. Which track, credited as a cover of the French new-wave group, appears as a European-bonus track...

Documents:

```

[Doc G] INDO LIVE is the third live album by French new wave
band Indochine... released in 1997... Disc One... Les tzars...
[Doc C] Eating disorders treatment and recovery... (distractor)
[Doc A] Anorexia Nervosa - Redemption Process... 8. Les Tzars
(Indochine Cover)... limited press of six thousand copies...
[Doc D] Radio Indochine is the second live album by French new
wave band Indochine... released in 1994... Les tzars - 5:12...

```

Answer: Les Tzars

Figure 2: **Search Agent Trajectory Compilation Example.** The top section shows the original question and ground truth answer. The middle section shows the original agentic trajectory (documents visited are highlighted in blue, documents returned by search but never visited are highlighted in red). The bottom section shows the ACC compiled QA. Examples for SWE and SQL agents are provided in Appendix A.

4 Experiments

4.1 Experimental Setup

Base Model. We use Qwen3-30B-A3B-Thinking [25] as our base model.

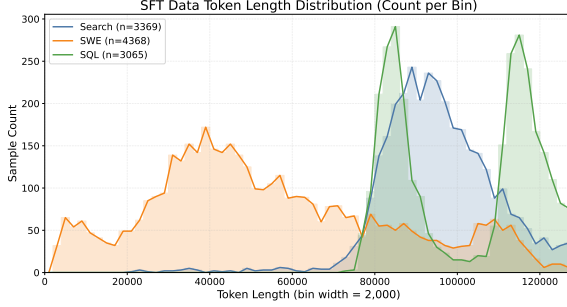


Figure 3: **Token length distribution of the ACC training data.** We bin the samples by token count and plot the per-bin frequency for the training data compiled from each agent type.

Table 1: Training parameters for our supervised fine-tuning.

Hyperparameter	Value
Sequence length	131,072 tokens
Global batch size	16
Learning rate	1×10^{-5} (min 1×10^{-6})
LR schedule	Cosine with 5% warmup
Optimizer	AdamW ($\beta_1=0.9$, $\beta_2=0.999$, weight decay 0.1)
Loss	Cross-entropy (chunk size 1024)
Sequence parallelism	8
Expert parallelism	1
Training epochs	4

Table 2: Long-range dependency modeling benchmark results (avg@3). Numbers in parentheses show improvement over the Qwen3-30B-A3B-Thinking baseline. All results use default inference configurations without manual tuning of reasoning effort.

Model	MRCR ¹			GraphWalks ²		
	2-needle	4-needle	Overall	Parents	BFS	Overall
Base Model						
Qwen3-30B-A3B-Thinking	61.84	38.41	50.19	71.19	68.47	69.92
Our Method						
Qwen3-30B-A3B-Thinking + ACC (Ours)	76.90 (+15.06)	59.57 (+21.16)	68.28 (+18.09)	81.50 (+10.31)	72.95 (+4.48)	77.51 (+7.59)
Strong Baselines						
Qwen3-235B-A22B-Thinking	74.98	59.96	67.51	78.53	74.45	76.63
GPT-OSS-120B	46.72	29.16	38.00 ³	75.92	61.82	69.34
DeepSeek-V3.2-Thinking	81.60	60.32	71.01	89.87	80.26	85.39
GPT-5.1-Thinking	73.75	47.93	60.91	81.41	29.41	57.14
GLM-4.6-Thinking	73.63	55.33	64.53	80.29	77.41	78.95
Kimi-K2-Thinking	68.01	47.96	58.05	84.34	75.17	80.04

¹MRCR reports overall and sub-task (2-needle, 4-needle) scores, following the evaluation setting in Qwen-Long-L1.5 [18].

²GraphWalks reports overall and sub-task (Parents, BFS) precision, following the evaluation setting in LongCat-Flash-Omni [20].

³GPT-OSS-120B was evaluated using the HuggingFace checkpoint via vLLM, and its lower MRCR scores largely result from frequent harmony-format parsing failures when processing multi-turn inputs.

Training Configuration. We compile 10,802 trajectories in total (Search: 3,369; SWE: 4,368; SQL: 3,065), with compiled context lengths ranging from 2K to 128K tokens and distinct per-agent length distributions (Figure 3). The details of training parameters are summarized in Table 1.

Evaluation Benchmarks. We primarily evaluate on long-range dependency modeling benchmarks including MRCR [14] (multi-round coreference resolution) and GraphWalks [14] (graph traversal), which require tracking long-range relational dependencies across extended contexts. We also monitor general capabilities on GPQA-Diamond [17], MMLU-Pro [24], AIME [1], and IFEval [29] to check for negative transfer.

4.2 Main Results

Table 2 presents our main results on long-range dependency modeling benchmarks. On MRCR, ACC improves both the 2-needle and 4-needle settings, yielding an overall score of 68.28 (+18.09). On GraphWalks, ACC improves both the Parents and BFS sub-tasks, yielding an overall precision of 77.51 (+7.59). These results are comparable to Qwen3-235B-A22B on these long-range dependency modeling benchmarks despite having nearly $8\times$ fewer active parameters. For completeness, we also report results on additional long-context benchmarks in Appendix B.

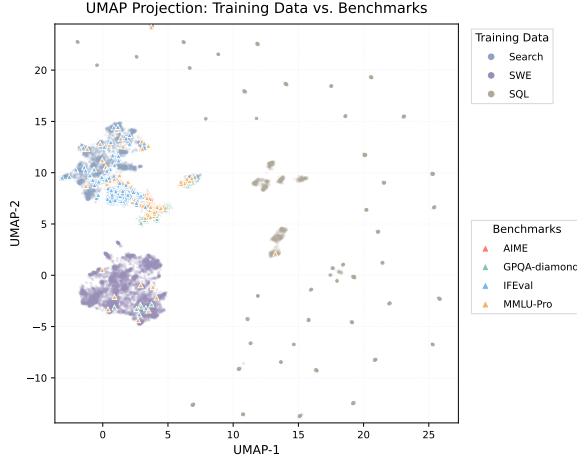


Figure 4: Two-dimensional UMAP projection of training queries (Search, SWE, SQL) and evaluation benchmark questions. Both training and evaluation samples are represented by their question text only.

Table 4: Quantitative separation between training queries and benchmark questions. Lower nearest-neighbor similarity and higher center distance both indicate limited overlap.

Benchmark	NN Sim.	Center Dist.
AIME	0.2832	0.8701
GPQA-Diamond	0.3557	0.7150
MMLU-Pro	0.3216	0.7685
IFEval	0.3425	0.9216
Overall AUC = 0.9986		

4.3 General Capability Preservation

Long-context training often raises concerns about negative transfer to general capabilities. As shown in Table 3, our ACC-trained model achieves slight improvements on GPQA-Diamond (+2.49), MMLU-Pro (+1.50) and AIME’25(+3.33), while performance on AIME’24 and IFEval remains stable. These results suggest that ACC does not introduce noticeable degradation to general abilities.

Table 3: General capability evaluation (avg@3). No significant negative transfer is observed.

Model	GPQA-Diamond	MMLU-Pro	AIME’24	AIME’25	IFEval
Base Model					
Qwen3-30B-A3B-Thinking	67.71	74.50	90.00	86.67	86.69
Our Method					
Qwen3-30B-A3B-Thinking + ACC (Ours)	70.20 (+2.49)	76.00 (+1.50)	90.00 (0.00)	90.00 (+3.33)	86.14 (−0.55)

To verify that these gains do not reflect test-set leakage, we compare the semantic distribution of training queries against benchmark questions. For each trajectory, we extract only the user question, stripping retrieved documents, code files, and database tables. Benchmark questions are similarly cleaned. Figure 4 shows the UMAP projection, and Table 4 reports quantitative metrics. Full details are in Appendix C.

The Search subset partially overlaps with general-knowledge benchmarks. Our multi-hop Search queries are synthesized from Wikipedia corpora, which naturally share topical vocabulary with knowledge benchmarks. The SWE and SQL subsets form distinct clusters. Quantitative analysis confirms this is domain-level overlap rather than instance duplication. The average nearest-neighbor cosine similarity remains below 0.36, and a linear classifier achieves an AUC of 0.9986 in separating training queries from benchmark questions. These patterns suggest the gains reflect transferable reasoning rather than data leakage.

4.4 Comparison with Long-Context Post-Training Methods

Table 5 compares ACC with recent long-context post-training methods. QwenLong-L1.5 [18] leads on MRCC through a multi-stage pipeline involving document cleaning, knowledge-graph construction, and RL. ACC surpasses it on GraphWalks while requiring only standard SFT. LongPO [4] and LongRLVR [5] release models trained on the Qwen2.5 and are listed for reference.³

³LoongRL [23] does not release trained checkpoints, so we do not include it in the comparison.

Table 5: Comparison with long-context post-training methods.

Model	MRCR	GraphWalks
Base Model		
Qwen3-30B-A3B-Thinking	50.19	69.92
Comparison Methods		
QwenLong-L1.5-30B ¹	92.30	73.85
Qwen2.5-7B-LongRLVR	19.76	15.72
Qwen2.5-14B-LongRLVR	20.06	22.78
Qwen2.5-7B-LongPO-128K ²	31.50	12.97
Our Method		
+ ACC (Ours)	68.28	77.51

¹QwenLong-L1.5 is trained with an agent framework that is not publicly available, so we evaluate it with standard inference for contexts within 256K.

²LongPO checkpoint supports up to 128K context, and test instances exceeding this limit are excluded from evaluation.

Table 6: Agent-type and distractor ablations.

Training Data	MRCR	GraphWalks
Base Model		
Qwen3-30B-A3B-Thinking	50.19	69.92
Ablations		
+ Search (Agent SFT)	42.16 (-8.03)	57.87 (-12.05)
+ Search	58.33 (+8.14)	44.75 (-25.17)
+ Search (w/o distractor)	54.99 (+4.80)	58.46 (-11.46)
+ SWE	54.82 (+4.63)	50.66 (-19.26)
+ SWE (w/o distractor)	51.01 (+0.82)	52.88 (-17.04)
+ SQL	56.44 (+6.25)	75.50 (+5.58)
Our Method		
+ ACC (Ours)	68.28 (+18.09)	77.51 (+7.59)

4.5 Ablation Study

Agent-type ablation. Raw search trajectories with Agent SFT (observations masked) underperform the base model, confirming the **supervision blind spot** in Section 3.1. As shown in Table 6, we ablate ACC by training on each agent type separately. All single-agent variants improve over the baseline on MRCR (Search +8.14, SWE +4.63, SQL +6.25), indicating that compiling scattered evidence into a single context alone improves cross-turn coreference resolution. On GraphWalks, however, only SQL improves (+5.58), while Search and SWE fall behind. This gap likely reflects differences in evidence structure. SQL tables are inherently relational and suit graph traversal, whereas web pages and source files are longer continuous passages that make discrete node-level reasoning harder to learn. The full mixture surpasses all single-agent variants, showing that diverse trajectory types offer complementary coverage.

Distractor ablation. Removing distractors from Search and SWE lowers MRCR by 3.34 and 3.81 points, confirming that including unvisited results and unopened files in the compiled context helps the model to learn localizing critical evidence. On GraphWalks, the single-agent setting shows the opposite trend, with Search and SWE without distractors gaining +13.71 and +2.22 respectively. This is because Search and SWE distractors are semantically unrelated to the query, helping the model learn noise filtering but offering little benefit for graph traversal. The full mixture, enriched by SQL’s relational data, benefits from distractors for localization while preserving graph-walking capability. The full ACC mixture still achieves the best overall result (77.51).

4.6 Mechanism Analysis

To understand how ACC improves long-range dependency modeling capacity, we visualize attention distance distributions and expert routing patterns on GraphWalks and MRCR examples.

Task-specific attention restructuring. Figure 5(a–b) shows attention distance distributions before and after ACC, with experimental settings detailed in Appendix D. On GraphWalks, the ACC-trained model shows increased relative attention mass at both nearby and far-distance bins, consistent with the task structure requiring local neighborhood checks and distant node jumps. On MRCR, the ACC-trained model shows higher relative attention mass at nearby distance bins while preserving the baseline long-range attention profile. The increased local focus indicates improved precision in verifying candidate segments during scanning. Notably, the three layers exhibiting the largest attention changes differ completely between the two tasks. These distinct patterns suggest the ACC-trained model adjusts its attention allocation flexibly rather than following a fixed uniform pattern.

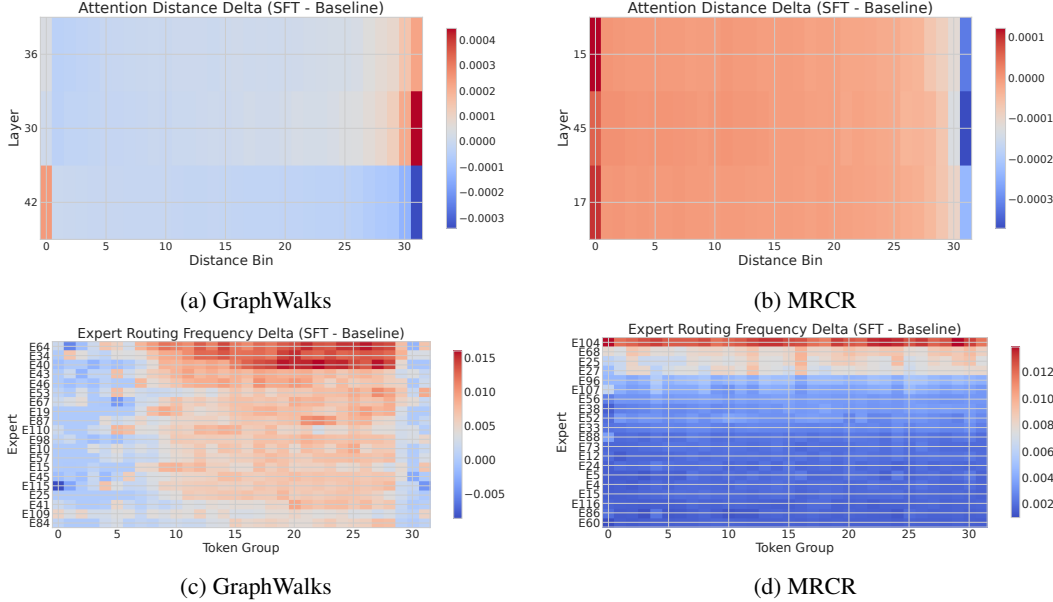


Figure 5: Attention distance (top) and expert routing frequency (bottom) changes after ACC training (SFT minus baseline). (a–b) Attention: GraphWalks shows increased mass at nearby and far-distance bins. MRCR shows enhancement primarily at nearby bins. (c–d) Expert routing: GraphWalks distributes activation across several experts for distant tokens. MRCR concentrates activation in a small expert set.

Expert specialization. Figure 5(c–d) shows changes in expert activation after ACC, with experimental settings detailed in Appendix E. On GraphWalks, higher activation for distant token groups is distributed across several experts, suggesting balanced processing of cross-node jumps. On MRCR, one expert shows much higher activation across all token groups while most others are suppressed, pointing to dedicated processing of scanning and verification. Notably, the layers with the strongest expert activation shifts are completely different across the two tasks. Both phenomena reflect task-dependent expert specialization after ACC training.

5 Conclusion

We presented Agent Context Compilation (ACC), a simple but effective method that compiles multi-turn agent trajectories into long-context training data. ACC complements existing long-context extension or training methods and can be combined with them. The ACC-trained Qwen3-30B-A3B achieves results comparable to Qwen3-235B-A22B on MRCR and GraphWalks, benchmarks that test long-range dependency modeling, while largely preserving general capabilities. Mechanistic analyses suggest task-specific attention restructuring and task-dependent expert specialization after ACC training. Future work includes extending ACC to more agent types and scaling to longer contexts.

6 Limitations and Social Impacts

ACC is evaluated on three agent types and one model, so broader generalization and scaling to million-token contexts remain to be studied. Reasoning synthesis depends on a strong teacher model, risking bias propagation. On the societal side, ACC lowers annotation costs by reusing agent logs, yet two risks should be noted. First, raw trajectories may leak private information without proper filtering. Second, compiled contexts may include copyrighted or proprietary material, raising intellectual property concerns. We recommend careful data filtering and safety alignment.

References

- [1] AIME. American invitational mathematics examination. <https://artofproblemsolving.com/wiki/index.php/AIME>.
- [2] Anthropic. Claude opus 4.6 system card. <https://www.anthropic.com/news/claude-opus-4-6>, 2026. Accessed: 2026-05-02.
- [3] Y. Bai, S. Tu, J. Zhang, H. Peng, X. Wang, X. Lv, S. Cao, J. Xu, L. Hou, Y. Dong, J. Tang, and J. Li. Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks, 2025.
- [4] G. Chen, X. Li, M. Q. Shieh, and L. Bing. Longpo: Long context self-evolution of large language models through short-to-long preference optimization, 2025.
- [5] G. Chen, M. Q. Shieh, and L. Bing. Longrlvr: Long-context reinforcement learning requires verifiable context rewards, 2026.
- [6] Google DeepMind. Gemini 3.1 pro. <https://deepmind.google/models/gemini/pro/>, 2026. Accessed: 2026-05-02.
- [7] C.-P. Hsieh, S. Sun, S. Krizan, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, and B. Ginsburg. Ruler: What’s the real context size of your long-context language models?, 2024.
- [8] J. Jia, X. Wu, C. Gao, Z. Chen, Z. Lin, Z. Li, W. Wang, H. Xu, D. Jin, D. Zhang, and B. Guo. Litelong: Resource-efficient long-context data synthesis for llms, 2025.
- [9] G. Kamradt. Llmtest (needle in a haystack). https://github.com/gkamradt/LLMTest_NeedleInAHaystack, 2023. Accessed: 2026-05-02.
- [10] T. Kočiský, J. Schwarz, P. Blunsom, C. Dyer, K. M. Hermann, G. Melis, and E. Grefenstette. The narrativeqa reading comprehension challenge, 2017.
- [11] A. Lahoti, K. Y. Li, B. Chen, C. Wang, A. Bick, J. Z. Kolter, T. Dao, and A. Gu. Mamba-3: Improved sequence modeling using state space principles, 2026.
- [12] X. Liu, Y. Song, Z. Liu, Z. Huang, Q. Guo, Z. Liu, S. Lian, Z. He, and X. Qiu. Beyond real: Imaginary extension of rotary position embeddings for long-context llms, 2025.
- [13] K. Lv, X. Liu, Q. Guo, H. Yan, C. He, X. Qiu, and D. Lin. Longwanjuan: Towards systematic measurement for long text quality, 2024.
- [14] OpenAI. Introducing GPT-4.1, 2025. Accessed: 2026-05-02. Introduces the MRCR and GraphWalks benchmarks.
- [15] OpenAI. Gpt-5.4. <https://openai.com/index/introducing-gpt-5-4/>, 2026. Accessed: 2026-05-02.
- [16] Qwen Team. Qwen3.5. <https://qwen.ai/blog?id=qwen3.5>, 2026. Accessed: 2026-05-02.
- [17] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. Gpqa: A graduate-level google-proof qa benchmark, 2023.
- [18] W. Shen, Z. Yang, C. Li, Z. Lu, M. Peng, H. Sun, Y. Shi, S. Liao, S. Lai, B. Zhang, D. Liu, F. Huang, J. Zhou, and M. Yan. Qwenlong-ll.5: Post-training recipe for long-context reasoning and memory management, 2025.
- [19] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han. Quest: Query-aware sparsity for efficient long-context llm inference, 2024.
- [20] M. L. Team, B. Wang, Bayan, B. Xiao, B. Zhang, B. Rong, B. Chen, C. Wan, C. Zhang, C. Huang, C. Chen, C. Chen, C. Yang, C. Yang, C. Han, D. Peng, D. Ruan, D. Xin, D. Wang, D. Yang, F. Liu, F. Chen, F. Yang, G. Dong, G. Huang, G. Xu, G. Wan, G. Tan, G. Yu, H. Qiu, H. Lu, H. Liu, H. Xiang, J. Wu, J. Yang, J. Liu, J. Huang, J. Wang, J. Ding, J. Jiang, J. Kuang, J. Wang, J. Mei, K. Ding, K. Zhang, L. Chen, L. Shi, L. Qiao, L. Zheng, L. Ma, L. Guo, L. Ma,

- L. Sun, M. Gao, M. Zhu, M. Cao, M. Lin, N. Xu, P. Shi, Q. Zhang, Q. Fang, Q. Wang, Q. Yang, Q. Wang, R. Weng, R. Guo, R. Liang, S. Yang, S. Xu, S. Lei, S. Ye, S. Chen, S. Chen, S. Hu, S. Li, S. Yang, S. Xu, S. Ren, S. Li, S. Liu, T. Bai, T. Dai, W. Hong, W. Wang, W. Zhao, W. Cao, W. Zhu, W. He, X. Su, X. Nan, X. Zhao, X. Wang, X. Zhao, X. Wang, X. Li, X. Pan, X. Chen, X. Sun, X. Xiang, X. Xing, X. Cao, X. Cai, Y. Yang, Y. Tan, Y. Yao, Y. Sun, Y. Chen, Y. Lu, Y. Gong, Y. Zhang, Y. Chen, Y. Gan, Y. Tang, Y. Xie, Y. Wang, Y. Zheng, Y. Zhang, Y. Zhong, Y. Qian, Y. Peng, Y. Li, Y. Jiang, Z. Hu, Z. Zhang, Z. Tian, Z. Hong, Z. Zeng, Z. Mi, Z. Li, Z. Wang, Z. Zhao, Z. Zhuang, and Z. Zhao. Longcat-flash-omni technical report, 2025.
- [21] Q. Tian, W. Zhu, X. Liu, X. Wang, and R. Wang. Mrrope: Mixed-radix rotary position embedding, 2026.
- [22] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal. Musique: Multihop questions via single-hop question composition, 2022.
- [23] S. Wang, G. Zhang, L. L. Zhang, N. Shang, F. Yang, D. Chen, and M. Yang. Loongrl: Reinforcement learning for advanced reasoning over long contexts, 2025.
- [24] Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang, T. Li, M. Ku, K. Wang, A. Zhuang, R. Fan, X. Yue, and W. Chen. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark, 2024.
- [25] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li, M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, and Z. Qiu. Qwen3 technical report, 2025.
- [26] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering, 2018.
- [27] H. Yu, T. Chen, J. Feng, J. Chen, W. Dai, Q. Yu, Y.-Q. Zhang, W.-Y. Ma, J. Liu, M. Wang, and H. Zhou. Memagent: Reshaping long-context llm with multi-conv rl-based memory agent, 2025.
- [28] J. Yuan, H. Gao, D. Dai, J. Luo, L. Zhao, Z. Zhang, Z. Xie, Y. X. Wei, L. Wang, Z. Xiao, Y. Wang, C. Ruan, M. Zhang, W. Liang, and W. Zeng. Native sparse attention: Hardware-aligned and natively trainable sparse attention, 2025.
- [29] J. Zhou, T. Lu, S. Mishra, S. Brahma, S. Basu, Y. Luan, D. Zhou, and L. Hou. Instruction-following evaluation for large language models, 2023.

Appendix

A Agent Trajectory Compilation Examples

Figures 6 and 7 show compiled trajectories for SWE and SQL agents. Both follow the same ACC pipeline as the search agent in Figure 2.

Figure 6 shows a compiled trajectory for the SWE agent. The environment presents a partial codebase snapshot containing both files relevant to the bug and irrelevant distractors. The agent opens files selectively to locate the issue, and ACC compiles the opened file contents into a long-context background while shuffling in unopened distractors.

Figure 7 shows a compiled trajectory for the SQL agent. The environment presents a relational table that encodes a multi-hop graph structure. In the original trajectory, the agent issues SQL queries to perform recursive traversals. ACC compiles the full contents of the relevant table into a long-context background, enabling the model to perform multi-hop relational reasoning directly over the assembled records without SQL query execution.

B Extended Results on General Long-Context Tasks

Table 7 reports results on general long-context benchmarks, including multi-hop QA (HotpotQA[26], MuSiQue[22]), long-document understanding (NarrativeQA[10]), and comprehensive long-context suite (LongBench-V2[3]). ACC yields modest gains on these tasks.

Table 7: Extended long-context benchmark results (avg@3). Numbers in parentheses show improvement over the Qwen3-30B-A3B-Thinking baseline.

Model	LB-V2	HotpotQA	MuSiQue	NarrQA
Base Model				
Qwen3-30B-A3B-Thinking	47.87	87.00	70.10	85.00
Our Method				
Qwen3-30B-A3B-Thinking + ACC (Ours)	48.90 (+1.03)	88.50 (+1.50)	70.40 (+0.30)	85.50 (+0.50)
Strong Baseline				
Qwen3-235B-A22B-Thinking	59.76	90.00	73.30	88.50

C Data Overlap Experiment Details

Question Extraction. For each training trajectory, we parse the dialogue and keep only the user turns. We then apply a lightweight rule-based extractor to obtain the core question. Benchmark questions undergo the same cleaning pipeline to retain only the problem statement. The extracted questions are whitespace-normalized and truncated to 3,000 characters as a safety bound, though most questions are far shorter after extraction.

Embedding. We encode all cleaned questions with `all-MiniLM-L6-v2`. The model outputs 384-dimensional vectors, which we normalize to unit length. Cosine similarity is used for all distance computations. The model processes at most 256 tokens internally, so the effective input is the leading segment of each extracted question.

Dimensionality Reduction. We project the embeddings into two dimensions using UMAP with the following fixed settings: 15 nearest neighbors, minimum distance 0.3, cosine metric, PCA initialization, and random seed 42. The PCA initializer avoids the spectral initialization failure that can occur on densely connected graphs.

Metrics. All quantitative indicators are computed on the original 384-dimensional embeddings. UMAP coordinates are used only for visualization.

SWE Agent Trajectory Compilation

Original Question-Answer Pair:

Question: Fix the slow transpilation of ExcitationPreserving to basis gates u1, u2, u3, cx. Transpilation time grows from 0.8 s (6 qubits) to over 3988 s (24 qubits).

Groundtruth: Wrap global_phase additions with sympy evaluate(False) in basis_translator.py and dagcircuit.py.

Original Agentic Trajectory (abridged):

```
Turn 1 (Open): qiskit/transpiler/passes/basis/basis_translator.py
Content: [File A] ... dag.global_phase +=
          bound_target_dag.global_phase ... in run() ...
-> (Examine) File A

Turn 2 (Open): qiskit/dagcircuit/dagcircuit.py
Content: [File B] ... self.global_phase +=
          in_dag.global_phase ... in substitute_node_with_dag() ...
-> (Examine) File B

Turn 3 (Open): qiskit/transpiler/passes/basis/basis_translator.py
Content: [File A] ... dag.global_phase +=
          bound_target_dag.global_phase ...
-> (Modify) File A
      from sympy import evaluate
      with evaluate(False):
          dag.global_phase += bound_target_dag.global_phase

Turn 4 (Open): qiskit/dagcircuit/dagcircuit.py
Content: [File B] ... self.global_phase +=
          in_dag.global_phase ...
-> (Modify) File B
      from sympy import evaluate
      with evaluate(False):
          self.global_phase += in_dag.global_phase

Turn 5 (Submit): Tests passed ✓
```

ACC-Compiled Question-Answer Pair (abridged):

```
Question: Fix the slow transpilation of ExcitationPreserving to
basis gates u1, u2, u3, cx...

Context:
[File B] qiskit/dagcircuit/dagcircuit.py ...
self.global_phase += in_dag.global_phase ... in
substitute_node_with_dag() ...
[File A] qiskit/transpiler/passes/basis/basis_translator.py ...
dag.global_phase += bound_target_dag.global_phase ... in run() ...
[File C] qiskit/pulse/instructions/phase.py ...
(distractor: present in codebase snapshot but never opened)

Answer:
<patch>
--- a/qiskit/dagcircuit/dagcircuit.py
+++ b/qiskit/dagcircuit/dagcircuit.py
@@ -990,7 +990,9 @@
         if in_dag.global_phase:
-            self.global_phase += in_dag.global_phase
+            from sympy import evaluate
+            with evaluate(False):
+                self.global_phase += in_dag.global_phase
... (analogous change in basis_translator.py)
</patch>
```

Figure 6: **SWE Agent Trajectory Compilation Example.** The top section shows the original question and ground truth answer. The middle section shows the original agentic trajectory. At each turn the agent opens a single file from the provided codebase snapshot and decides either to (Examine) it for understanding or to (Modify) it to fix the bug. The bottom section shows the ACC-compiled QA, where only the *opened* evidence is retained and an irrelevant distractor (a file present in the snapshot but never opened, highlighted in red) is shuffled into the provided long-context background.

- **Average Nearest-Neighbor Cosine Similarity.** For each benchmark question, we identify the most similar training sample by cosine similarity and average these maxima across all benchmark instances.

SQL Agent Trajectory Compilation

Original Question-Answer Pair:

Question: User 'u_337bc99c' (Region: East) has applied for Diamond Membership. Verify their complete referral lineage by walking upward through the referral graph following active edges (status='active') with edge_level >= 3 within region 'East'. Prioritize higher commission_rate at each step; break ties by alphabetical referrer_id. Report the final root referrer.

Groundtruth: u_ea8952bc

Original Agentic Trajectory (abridged):

```
Turn 1 (Execute SQL):
WITH RECURSIVE upstream AS (
  SELECT referrer_id, referred_id, commission_rate,
         edge_level, status, region, 1 AS depth
  FROM referrals
  WHERE referred_id = 'u_337bc99c' AND status = 'active'
    AND edge_level >= 3 AND region = 'East'
  UNION ALL
  SELECT r.referrer_id, r.referred_id, r.commission_rate,
         r.edge_level, r.status, r.region, u.depth + 1
  FROM upstream u
  JOIN referrals r ON r.referred_id = u.referrer_id
  WHERE r.status = 'active' AND r.edge_level >= 3
    AND r.region = 'East'
)
SELECT referrer_id FROM upstream
ORDER BY depth DESC LIMIT 1;

Result: u_ea8952bc

Turn 2 (Answer): u_ea8952bc ✓
```

ACC-Compiled Question-Answer Pair (abridged):

Question: User 'u_337bc99c' (Region: East) has applied for Diamond Membership. Walk upward through the referral graph following active edges with edge_level >= 3 within region 'East', prioritizing higher commission_rate. Report the final root referrer.

Context:

[Table: referrals]

	referrer_id	level	rate	ltv	region
...	u_4df289c8	5	0.15	82.19	East
...	u_fb9c918a	3	0.08	201.42	East
...	u_be13d04d	4	0.12	336.31	East
...	u_a4663191	5	0.15	264.93	East
...	u_e1ce60aa	3	0.05	443.63	East
...	u_54ef1623	3	0.05	355.49	East
...	u_ea8952bc	5	0.10	108.68	East
...	u_b7c33fc2	3	0.10	264.93	East
...	u_63490c55	3	0.05	201.42	North
...	u_d28bb4a9	1	0.12	254.94	East
...	u_8eda5bd1	3	0.08	254.94	Central
... (remaining rows omitted)					

Answer: u_ea8952bc

Figure 7: **SQL Agent Trajectory Compilation Example.** The top section shows the original question and ground truth answer. The middle section shows the original agentic trajectory, where the agent executes a recursive SQL query to traverse the referral graph. The bottom section shows the ACC-compiled QA, where the complete contents of the relevant database table are assembled into the provided long-context background. The ellipsis column (...) indicates additional fields that are present in the compiled context but omitted here for brevity.

- **Center Cosine Distance.** We compute the normalized mean embedding vector for the training set and for each benchmark, then take the cosine distance between these centroids (i.e., one minus their cosine similarity).

- **Linear Classifier AUC.** We train a logistic regression classifier to distinguish training samples from benchmark samples. We report the area under the ROC curve for the full training set and for the Search subset alone against all benchmarks.

D Attention Analysis Experiment Details

Setup. We analyze the baseline model (Qwen-30B-A3B-Thinking) and the ACC-trained checkpoint. Both models are loaded with `AutoModelForCausalLM` in `bfloat16` precision via `device_map="auto"`. To ensure the attention tensors are accessible, we force the attention implementation to eager mode, avoiding fused kernel paths that do not expose the full 4D attention matrix.

Layer and Distance Binning. We restrict the analysis to the three layers with the most significant attention changes for each task (indices 36, 30, 42 for GraphWalks and indices 15, 45, 17 for MRCR). For each head in these layers, we extract the causal attention matrix and bin token distances into 32 equal-width intervals ranging from 0 to the sequence length minus one. For each distance bin, we aggregate attention weights along the corresponding off-diagonals of the lower-triangular attention matrix and compute the per-head mean.

Metric Definition. For layer l and head h , let $A^{(l,h)} \in \mathbb{R}^{T \times T}$ denote the causal attention matrix where T is the sequence length. Let $[e_0, e_1, \dots, e_B]$ denote the bin edges where $B = 32$. For each distance bin b spanning $[e_b, e_{b+1})$, we aggregate attention weights along the lower-triangular off-diagonals (including the main diagonal at $d = 0$):

$$\mathcal{D}_b = \{d \in \mathbb{Z} : e_b \leq d < e_{b+1}\}.$$

The per-head per-bin mean is computed by averaging over all token positions that fall into those off-diagonals:

$$m_{l,h,b} = \frac{\sum_{d \in \mathcal{D}_b} \sum_{i=d}^{T-1} A_{i,i-d}^{(l,h)}}{\sum_{d \in \mathcal{D}_b} (T-d)}.$$

The per-layer per-bin mean is then obtained by averaging over all heads in the layer:

$$\mu_{l,b} = \frac{1}{H} \sum_{h=1}^H m_{l,h,b},$$

where H is the number of heads in layer l .

The reported heatmap shows the delta between the SFT and baseline models:

$$\Delta_{l,b} = \mu_{l,b}^{\text{SFT}} - \mu_{l,b}^{\text{Base}}.$$

Each cell in the heatmap corresponds to one layer-distance pair (l, b) . Positive values indicate increased attention mass at that distance after ACC training.

For the top-head analysis, we first compute the mean attention over the tail bins (the last 25% of distance bins) for each head:

$$\tau_{l,h} = \frac{1}{|B_{\text{tail}}|} \sum_{b \in B_{\text{tail}}} m_{l,h,b},$$

where B_{tail} indexes the farthest distance bins. The per-head tail delta is:

$$\delta_{l,h}^{\text{tail}} = \tau_{l,h}^{\text{SFT}} - \tau_{l,h}^{\text{Base}}.$$

This metric is used to rank and identify heads with the strongest far-range attention change.

Statistics and Visualization. Attention statistics are aggregated across evaluation samples and averaged per head and per distance bin.

E Expert Routing Visualization Experiment Details

Models and Layers. We compare the baseline model (Qwen-30B-A3B-Thinking) and the ACC-trained checkpoint. We restrict the analysis to three layers with the most significant expert routing changes for each task (indices 42, 40, 7 for GraphWalks and indices 17, 16, 15 for MRCR).

Dataset and Sampling. We randomly sample 32 examples from the evaluation splits of GraphWalks and MRCR, respectively. Each example is tokenized and fed through both models in inference mode to collect router statistics.

Metric Definition. For each token position t , layer l , and expert e , the router produces logits $z_{l,t} \in \mathbb{R}^E$ (E is the number of experts). Let $g_{l,t} \in \{0, 1\}^E$ be the top- k gating indicator where $g_{l,t,e} = 1$ if expert e is among the top- k selected experts for token t at layer l . We define the **top- k frequency** of expert e in token group i as

$$f_{l,e}^{(i)} = \frac{1}{|S_i|} \sum_{t \in S_i} g_{l,t,e},$$

where S_i is the set of token indices belonging to group i . Let $\mathcal{L}_{\text{task}}$ denote the set of three layers with the largest mean absolute expert routing delta for the target task. The reported heatmap shows the delta

$$\Delta f_e^{(i)} = \frac{1}{|\mathcal{L}_{\text{task}}|} \sum_{l \in \mathcal{L}_{\text{task}}} \left(f_{\text{SFT},l,e}^{(i)} - f_{\text{Baseline},l,e}^{(i)} \right),$$

where the per-layer expert frequency $f_{l,e}^{(i)}$ is averaged across the selected layers $l \in \mathcal{L}_{\text{task}}$.

Token Grouping. The full sequence is divided into 32 equal-length groups via linear binning of token indices (i.e., group i covers positions $[i \cdot L/32, (i+1) \cdot L/32)$). This relative-position grouping allows comparison across variable-length sequences.

Expert Selection. We rank experts by the mean absolute delta across all groups and layers, then visualize the top 20 experts with the largest change. This avoids cluttering the figure with experts whose routing patterns remain nearly unchanged after SFT.

Implementation. To collect router logits without modifying model weights, we temporarily wrap the MoE module’s forward pass in the selected three layers, extract the router logits during the forward pass, and immediately restore the original forward function. Statistics are aggregated incrementally across the 32 samples using running means to avoid storing large intermediate tensors.

F SWE Answer-Conditioned Rationale Generation

The original SWE trajectories contain file-open and edit actions but lack explicit natural-language reasoning. Direct rollout of CoT trajectories for SWE achieved only $\sim 10\%$ pass rate. To scale data construction, we generate answer-conditioned rationales by providing the compiled context C_i and the verified patch y_i to DeepSeek-V3.2-Thinking. The model is prompted to reconstruct a reasoning trace that grounds the patch in the evidence without revealing the patch content prematurely.

Figure 8 illustrates the complete prompt template and a generated example. We invoke the model with temperature 0.0 and enable its thinking mode via chat template configuration. To generate the reasoning trace, we prompt DeepSeek-V3.2-Thinking with the ACC-compiled question x_i together with the verified patch y_i . The model is instructed to reconstruct a reasoning process that grounds the patch in the evidence without revealing the patch content prematurely. We do not use the model’s internal reasoning content because it often contains meta-references to the provided ground-truth patch such as “We are given a ground-truth patch...” Instead we use only the final response content as the supervised trace r_i . The model simulates a discovery process spanning issue inspection, keyword search, file list browsing, distractor elimination, core file localization, logic analysis, and patch proposal. All generated rationales are retained directly without answer-correctness filtering because y_i is already verified. The final yield is effectively 100%.

SWE Answer-Conditioned Rationale Synthesis

Prompt Template (User Input to DeepSeek-V3.2-Thinking):

Please reason based on the following documents and answer the question. Show your complete step-by-step thought process, including issue inspection, keyword search, browsing the file list, eliminating distractors, locating core files, analyzing logic, and proposing modifications. Do not reveal the final patch directly in your reasoning trace.

Context:

```
[File B] qiskit/dagcircuit/dagcircuit.py
... self.global_phase += in_dag.global_phase ... in
    substitute_node_with_dag() ...
```

```
[File A] qiskit/transpiler/passes/basis/basis_translator.py
... dag.global_phase += bound_target_dag.global_phase ... in run() ...
```

```
[File C] qiskit/pulse/instructions/phase.py
... (distractor: present in codebase snapshot but never opened)
```

Question: Fix the slow transpilation of ExcitationPreserving to basis gates u1, u2, u3, cx. Transpilation time grows from 0.8s (6 qubits) to over 3988s (24 qubits).

```
Answer: <patch>
--- a/qiskit/dagcircuit/dagcircuit.py
+++ b/qiskit/dagcircuit/dagcircuit.py
@@ -990,7 +990,9 @@
     if in_dag.global_phase:
         self.global_phase += in_dag.global_phase
+        from sympy import evaluate
+        with evaluate(False):
             self.global_phase += in_dag.global_phase
     ... (analogous change in basis_translator.py)
</patch>
```

Generated Rationale (Final Response, used as r_i):

The issue describes a performance regression in transpilation time for ExcitationPreserving when targeting basis gates u1, u2, u3, cx. The slowdown suggests repeated symbolic evaluation of global_phase accumulations.

Looking at the provided files, File B (dagcircuit.py) contains `self.global_phase += in_dag.global_phase` in `substitute_node_with_dag()`. File A (basis_translator.py) has a similar pattern in `run()` where `dag.global_phase += bound_target_dag.global_phase`. Both operations accumulate SymPy expressions without wrapping them in `evaluate(False)`, causing exponential growth in symbolic complexity as qubit count increases.

File C (phase.py) is unrelated to the transpilation pass logic and can be ruled out as a distractor.

The fix should wrap both global_phase additions with SymPy's `evaluate(False)` context manager to defer symbolic evaluation. This matches the ground-truth modification provided in the answer.

Figure 8: SWE Answer-Conditioned Rationale Synthesis Example. Top: the prompt fed to DeepSeek-V3.2-Thinking, containing the compiled context C_i and the verified patch y_i . Bottom: the model’s final response, used as the supervised reasoning trace r_i .